

What we're doing

Implementing the training-data generator for `contacts-and-distances-v2`: serialization-identical to v1 with `` pause tokens spliced in between statements. Only generation in scope here — running at scale and training are deferred to follow-up issues.

This deck reports distributional statistics of the generated docs over a 100-protein representative sample, so we can sanity-check the document mechanics before scaling up.

Why

Pause tokens let a model spend extra forward-pass cycles without conditioning on additional content tokens (Goyal et al. 2023, arxiv 2310.02226). For v2 we needed to (a) preserve v1's algorithm end-to-end and (b) splice ``<think>`` runs without violating the 8192-token budget or breaking statement boundaries. The slides below confirm both.

Sample

100 FoldBench monomer ground-truth CIFs from
`exp20\_evals\_marifold\_1b\_foldbench/protenix\_data/data/protenix-foldbench-monomers/gt/`
(same set used by exp20 / exp26). Median length 222 residues, range 30..761.

Two caveats for absolute numbers:

1. These are PDB experimental structures, not AFDB predictions — length distribution and per-mode contact pools differ from AFDB-24M. 2. v1/v2's `residue\_plddt\_min` uses B-factor as a confidence proxy; the default 70 is calibrated for AFDB pLDDT (0..100) and would filter ~every PDB residue out. We disable the filter for this run (`--residue-plddt-min 0.0`) so the stats reflect document mechanics rather than the unrelated pLDDT-cutoff interaction with B-factor units.

The **shape** of the distributions here is what to read off; the absolute counts will shift on AFDB.

Headline: docs always pack to the 8192-token budget

Every generated doc lands within 5 tokens of 8192 (median 8190). The small deficit comes from integer division when budgeting distance statements after contacts have been placed — not an oversight.

→ See `**`06_context_fill.png`**`.

Headline: distance statements dominate the doc

Mean per-doc breakdown (n=100):

component	tokens	share
distance statements	7268	88.7%
contact statements (3 modes)	663	8.1%
residue tokens	246	3.0%
`<think>` tokens	8	0.1%
fixed overhead	5	0.1%

Distance statements consume almost 9/10 of every doc. That's the natural consequence of ``contact_f_range = (-0.1, 0.2)`` (per-mode fraction uniform with $E \approx 0.067$ after clamping), three modes, and distances claiming all remaining budget.

→ See `**`04_token_composition.png`**`.

Headline: medium- and short-range contacts almost always sa

Median fraction of *eligible* contacts captured in the doc, by mode:

- long-range: **0.41** (mean 0.46) - medium-range: **1.00** (mean 0.56) - short-range: **1.00** (mean 0.68)

For most proteins, the per-mode budget can hold every medium- and short-range contact in the eligible pool; long-range pools are far larger (mean ~381 eligible, max 1365) and only a fraction fits.

→ See **03_fraction_captured.png** and **08_eligible_vs_shown.png**.

Surprising: distance count ***falls*** with sequence length

Each scatter point in `**`05_statements_vs_length.png`**` is one document. Contact counts trend up with N (more eligible contacts), as expected. But ****distance statements per doc fall as the protein gets longer**** — from ~1320 on the smallest proteins to ~1000 on the largest. The mechanism is simple: residue tokens take a bigger bite of the 8192 budget at longer N, leaving fewer tokens for the statement appendix. Worth noting because a downstream eval that slices by length should expect this implicit reweighting.

→ See `**`05_statements_vs_length.png`**`.

Surprising: long-range pool grows ~super-linearly with N

Long-range eligible pool grows visibly faster than medium / short on a log-log plot. The ratio of long:medium:short flips around 100 residues: short proteins are short-range dominated; long proteins are long-range dominated. Combined with the bottleneck above, the per-mode budget bites hardest exactly where the *amount* of long-range information is largest.

→ See `**`09_eligible_pool_vs_length.png`**`.

`<think>` token cost is tiny and matches spec

- Initial gate fired on **81/100** docs (spec: $P=0.75$; well within sampling noise for $n=100$). - When fired, `k1`` empirical mean matches `Geom(0.13)`` ($E \approx 7.7$). - Additional-run counts 0/1/2/3 observed in 64/15/13/8 docs (spec: $P=0.625/0.125/0.125/0.125$, exactly). - Per-doc total mean: **8.0** `<think>` tokens ($\sim 0.1\%$ of context).

So `<think>` is essentially free relative to the rest of the doc. No correction to the v1 budget logic was needed — the pre-sampled overhead subtract works as intended.

→ See **`07_think_distribution.png``**.

Anything we should change before scaling up?

Nothing surfaced by this sample looks wrong. Two interpretive notes to keep in mind once we run on AFDB:

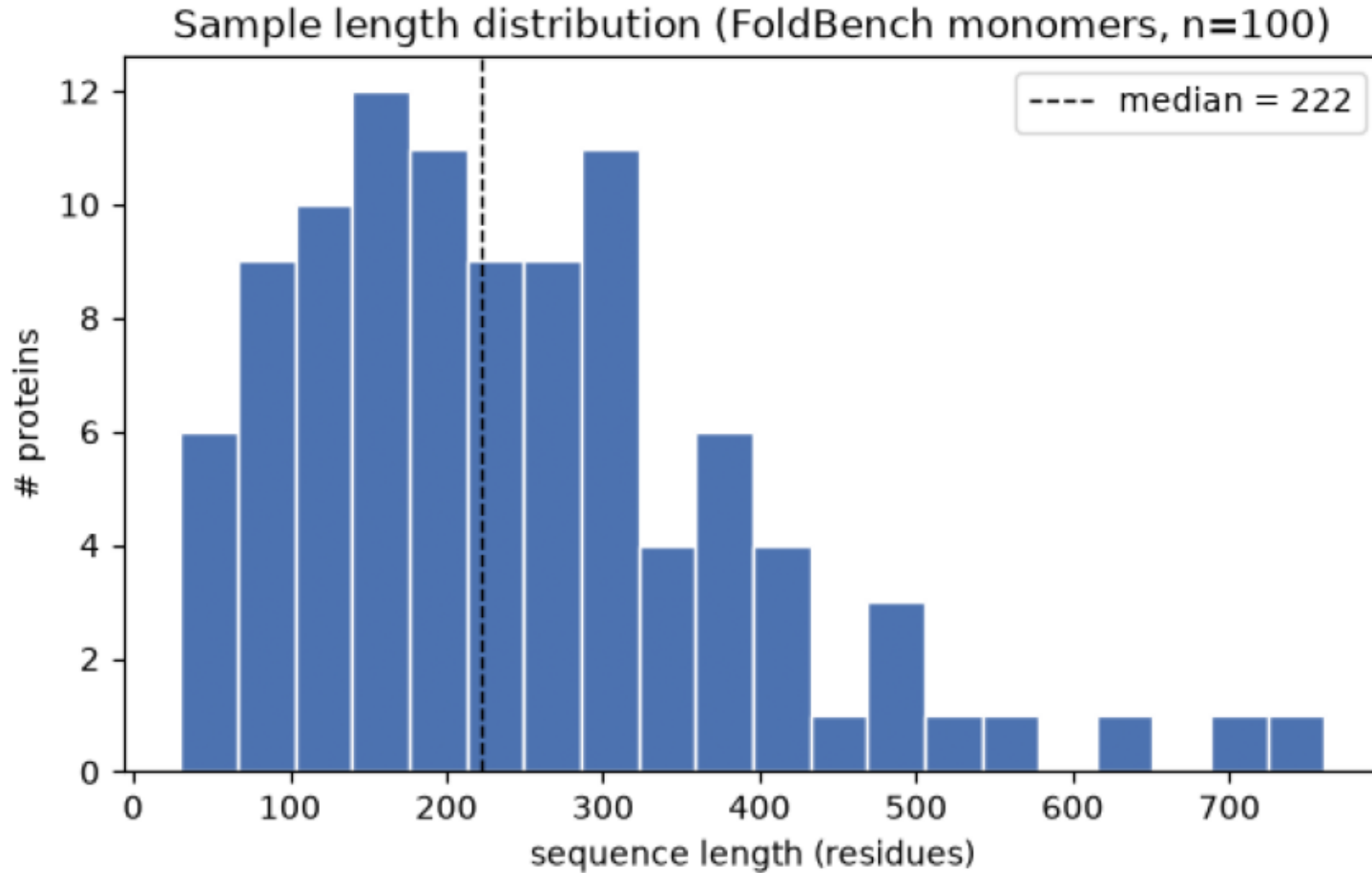
1. The pLDDT filter will eliminate ~half of low-confidence residues, shrinking the eligible-contact pools — so the long-range "fraction captured" will likely look **better** on AFDB than what we see here. (Smaller eligible pool → easier to saturate the per-mode budget.) Worth re-running this analysis on a few thousand AFDB structures once we have them. 2. Docs with zero contacts of any mode (the three-way clamp-to-0 case) happened in 4/100 here — close to the analytic 3.7% expectation. Not a defect; just something to keep an eye on when curating the published dataset.

What's next

Follow-up issues: (1) run this generator at scale on AFDB-24M (producing the v2 analogue of `contacts-and-distances-v1-5x`), (2) train a model on the result with loss masked at `` positions, (3) measure whether v2's pause-token training translates to a non-trivial accuracy boost at inference vs the v1 baseline.

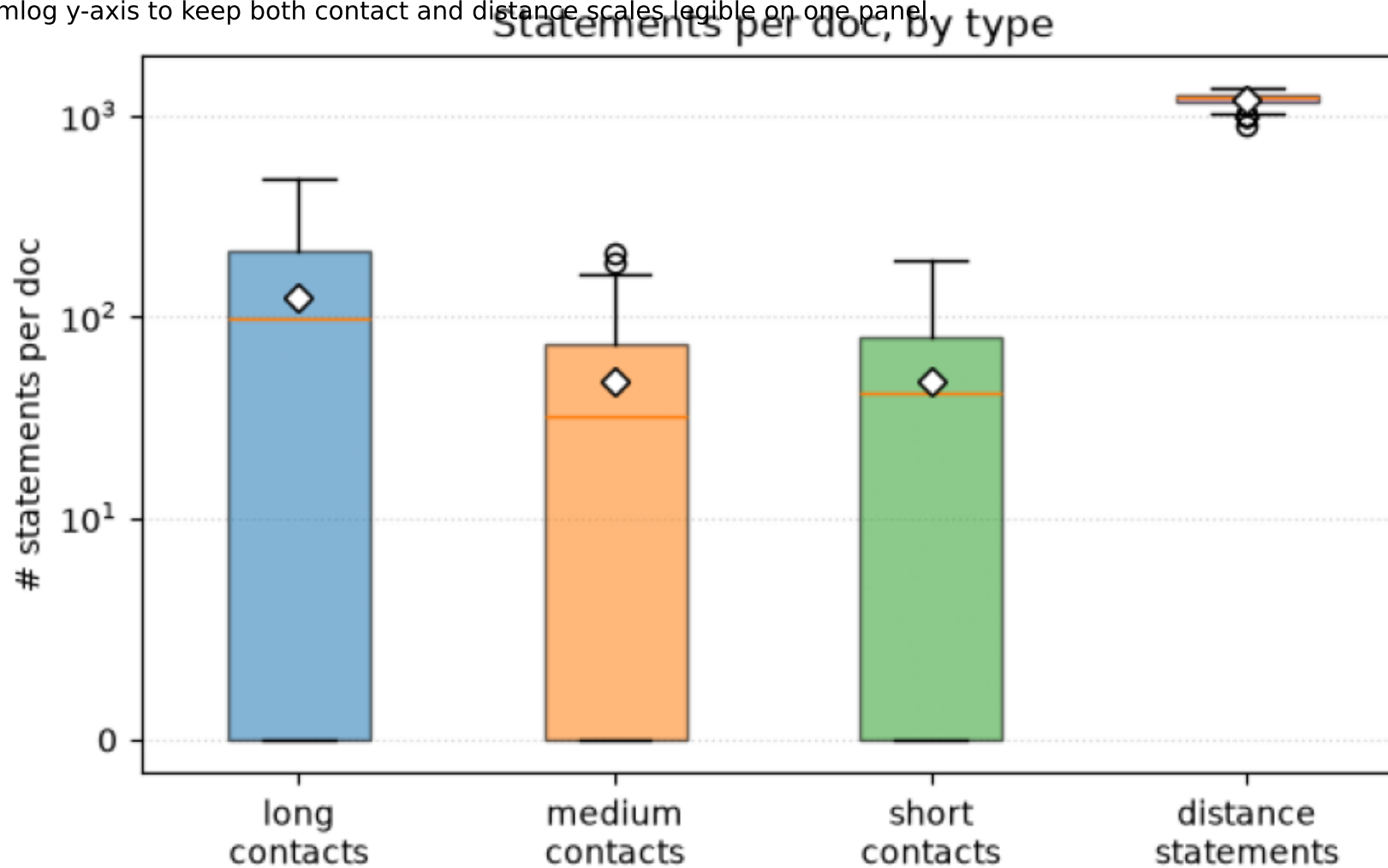
01_length_distribution.png

Sequence-length histogram of the 100 FoldBench monomer ground-truth CIFs used as the representative sample. Median ~ 220 residues; range 30..761.



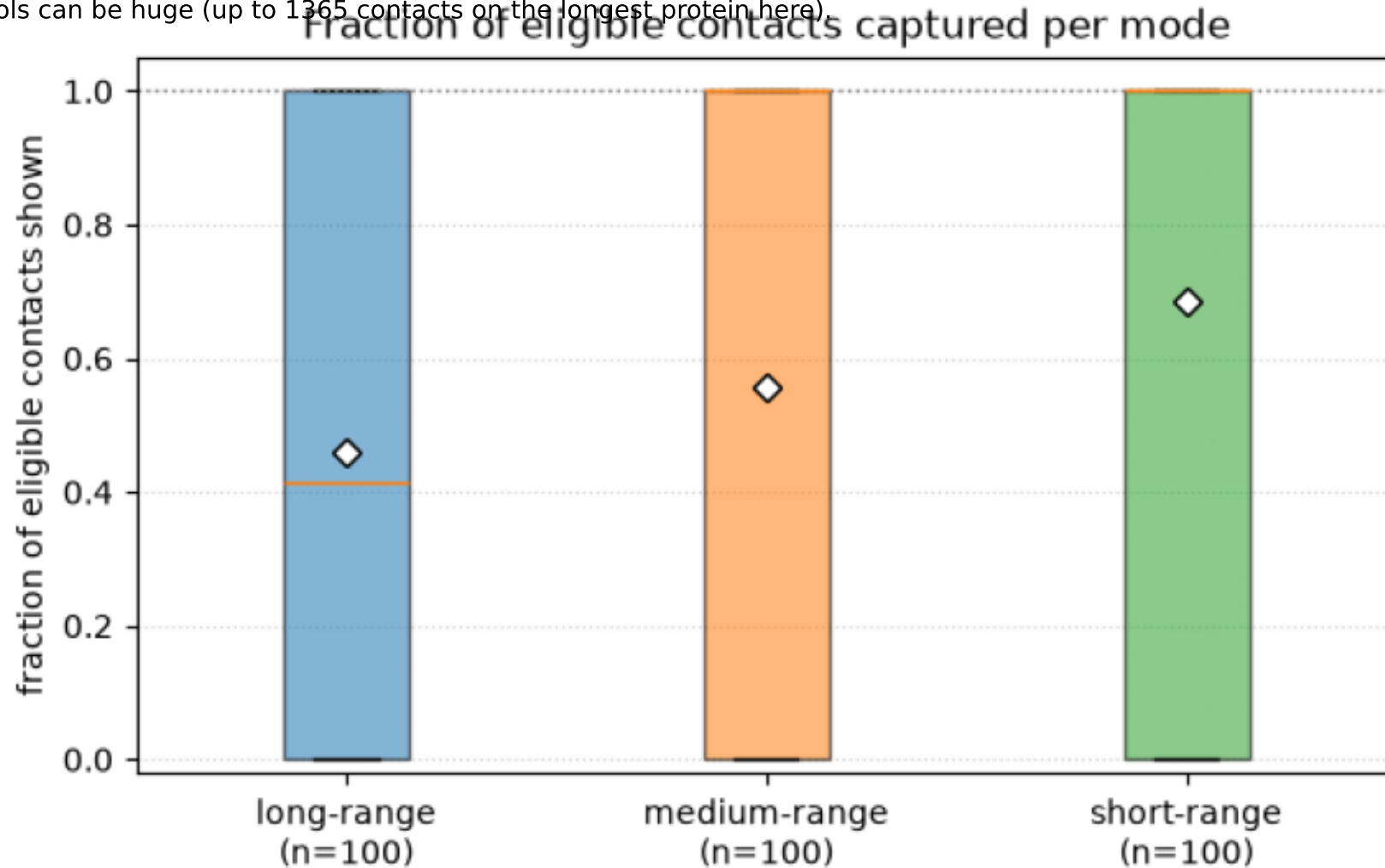
02_statements_per_doc.png

Statements per document, by type. Distance statements dominate (mean ~1210); contacts are limited by the per-mode budget knob ``contact\_f\_range`` (default uniform(-0.1, 0.2), clamped to 0). Long-range contacts are the most numerous because their eligible pool is the largest. Symlog y-axis to keep both contact and distance scales legible on one panel



03_fraction_captured.png

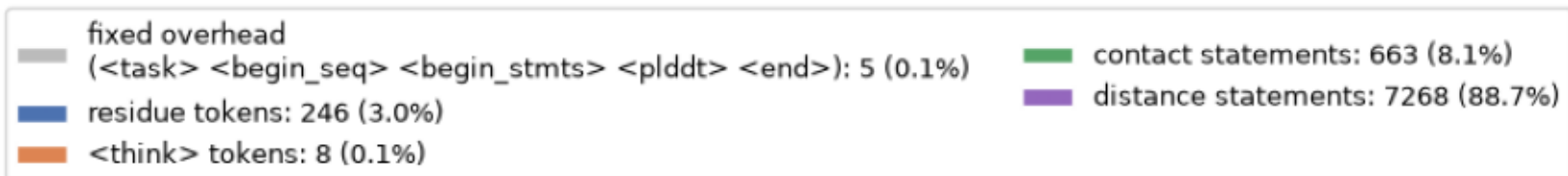
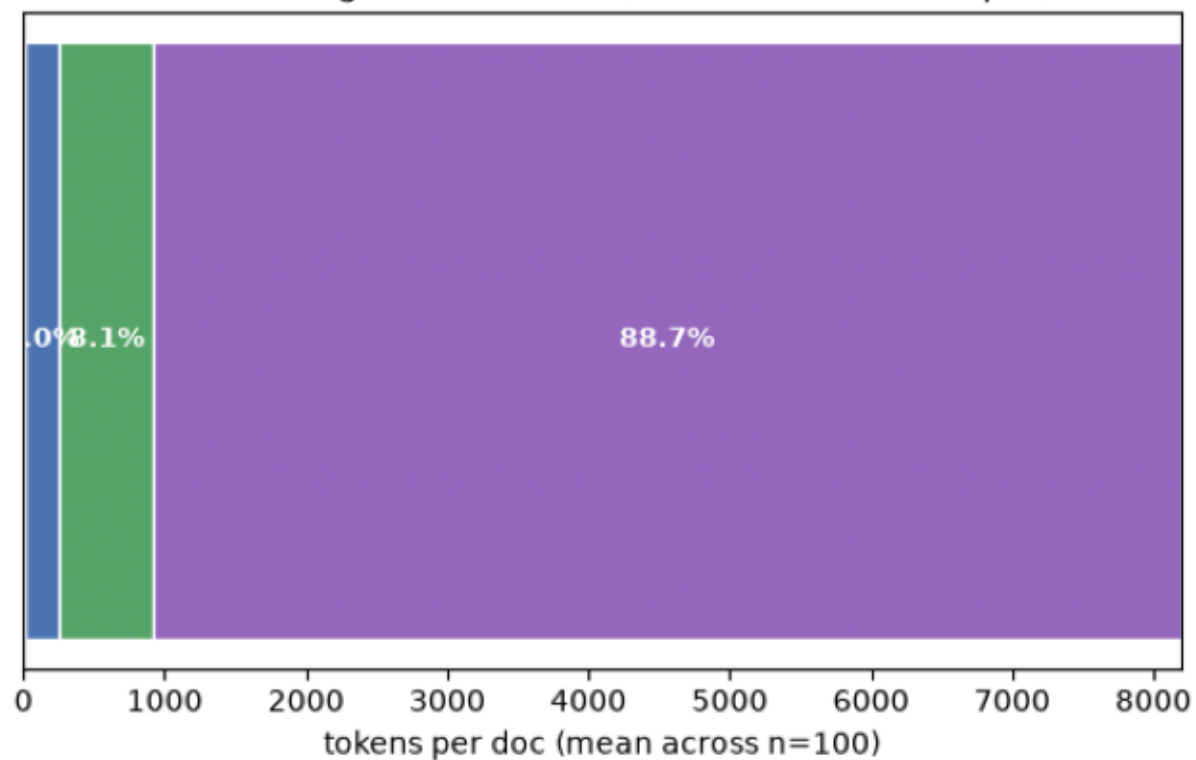
What fraction of a protein's eligible contacts make it into the doc, broken out by mode. Medium- and short-range typically saturate (median ~ 1.0) because the per-mode budget can usually hold them all. Long-range is the bottleneck — median ~ 0.41 , because long-range eligible pools can be huge (up to 1365 contacts on the longest protein here)



04_token_composition.png

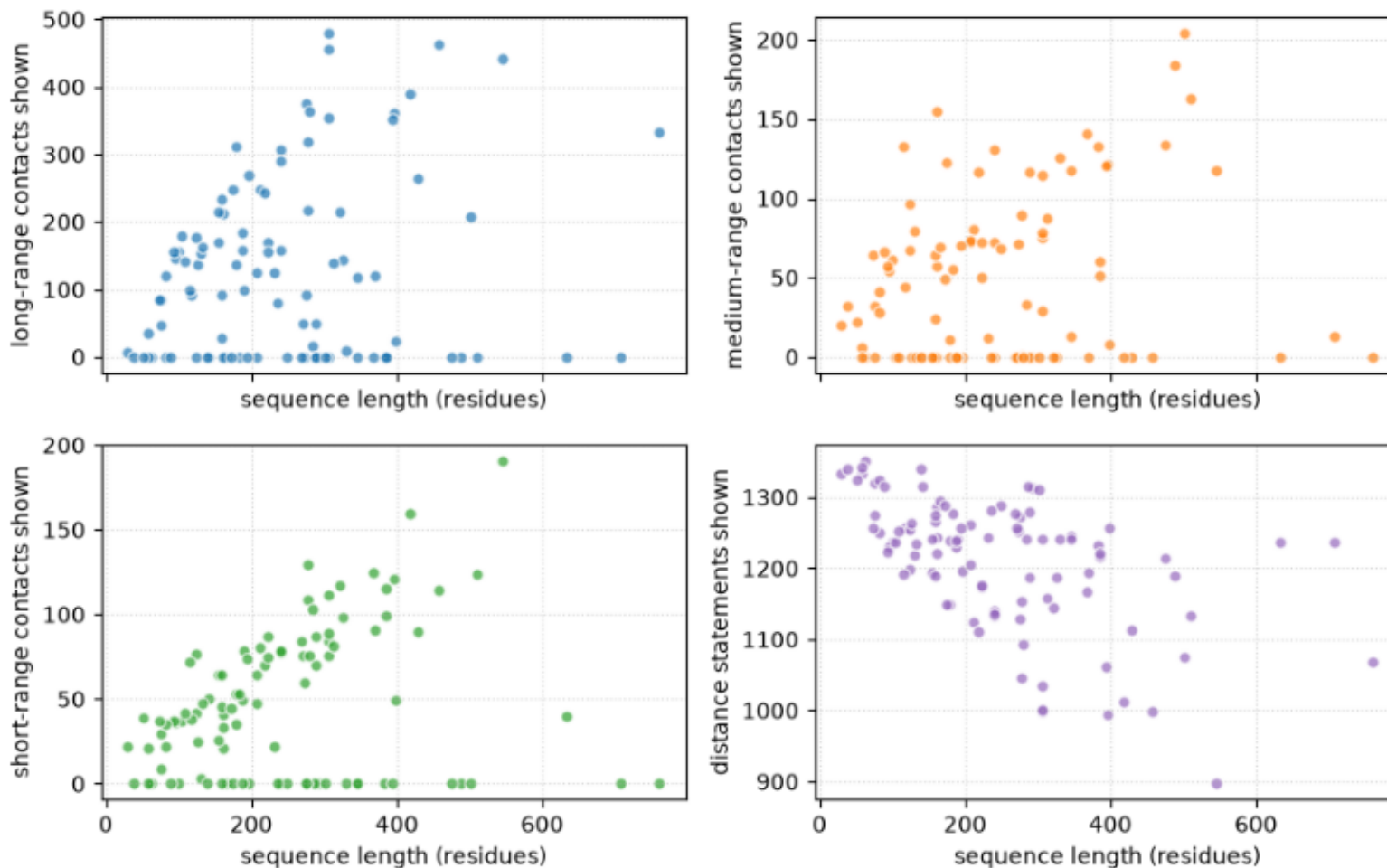
Per-doc token budget, averaged across the sample. Distance statements eat ~89% of the context (6 tokens each at ~1210 statements per doc); contacts ~8%, residues ~3%, <think> ~0.1%. Docs pack to the budget almost exactly (mean total ~8190/8192).

Token-budget breakdown (mean over the sample)



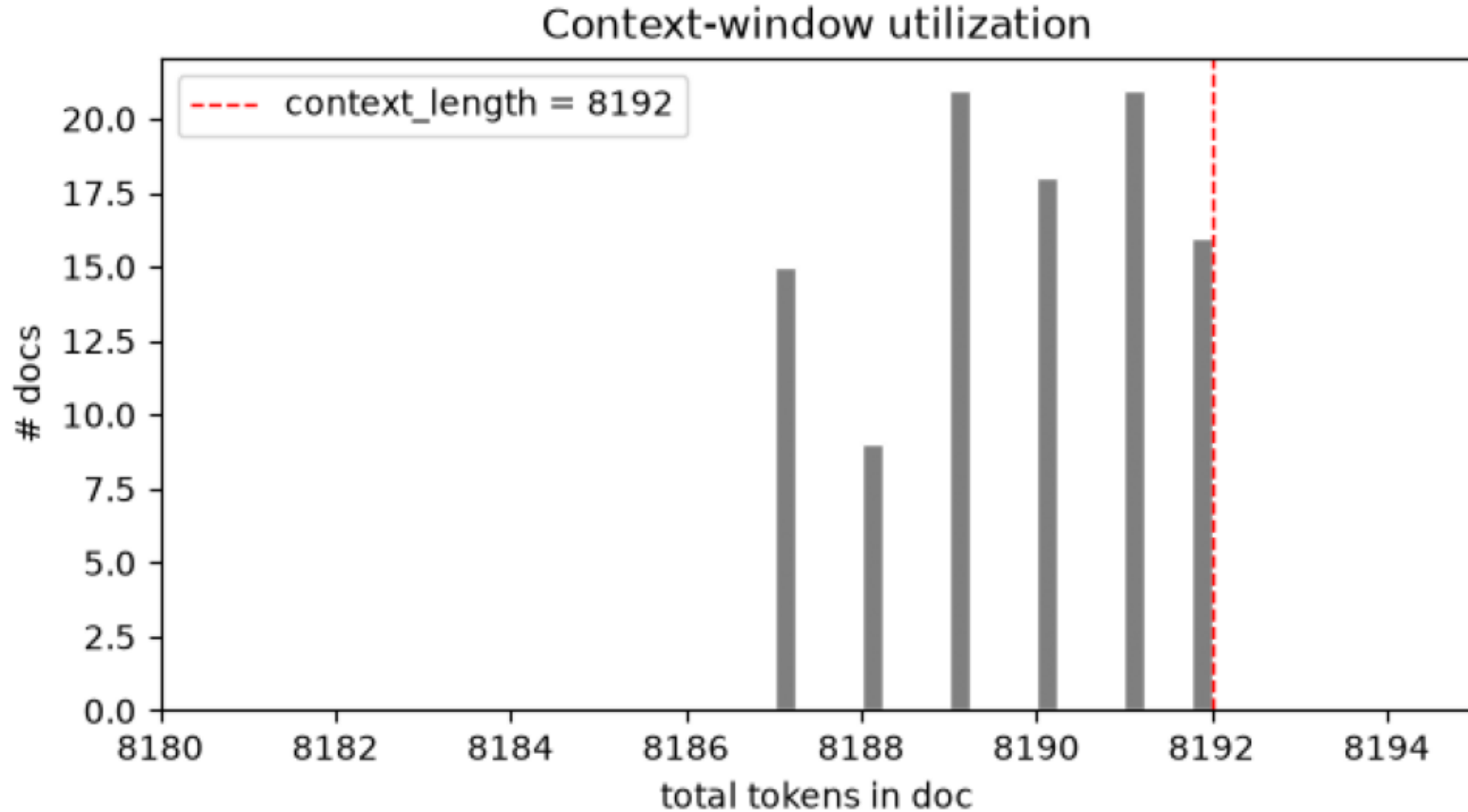
05_statements_vs_length.png

Each scatter point is one document. Contact-statement counts rise with sequence length (more eligible contacts available), with substantial variance from the per-mode fraction sampling. Distance-statement count *falls* with sequence length — because residue tokens take a larger bite of the 8192 budget, leaving less room for statements after contacts have claimed their share.



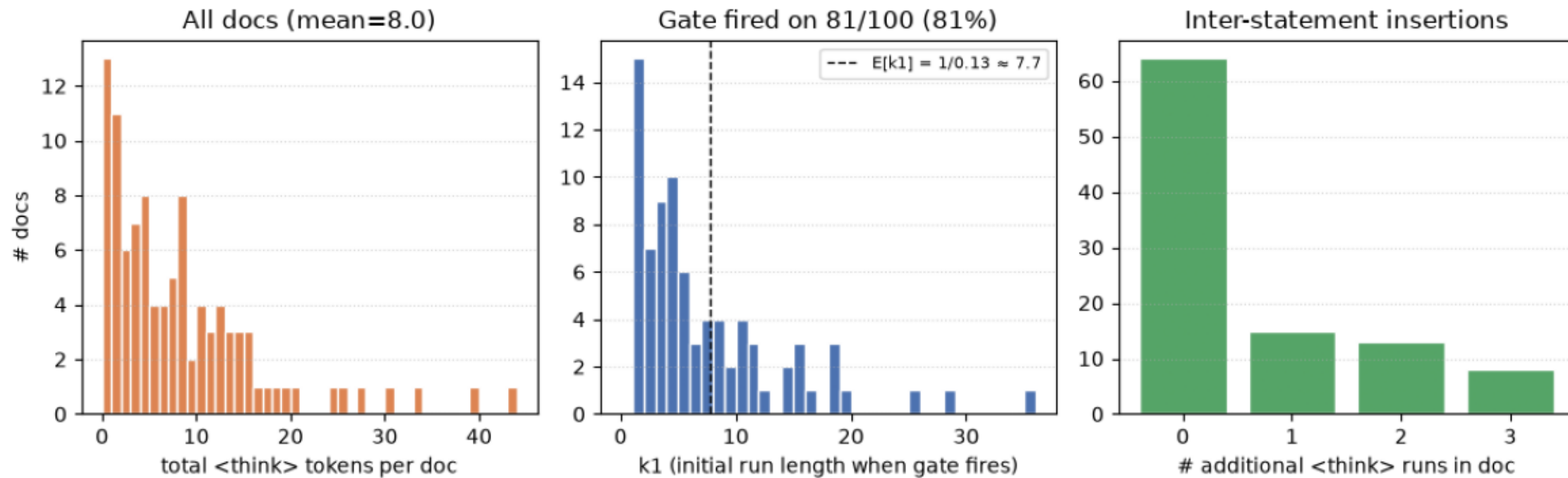
06_context_fill.png

Every doc fills the 8192-token budget within a handful of tokens. The small deficit (max=8192, median=8190) comes from integer-division when allocating distance statements out of the remaining budget after contacts are placed.



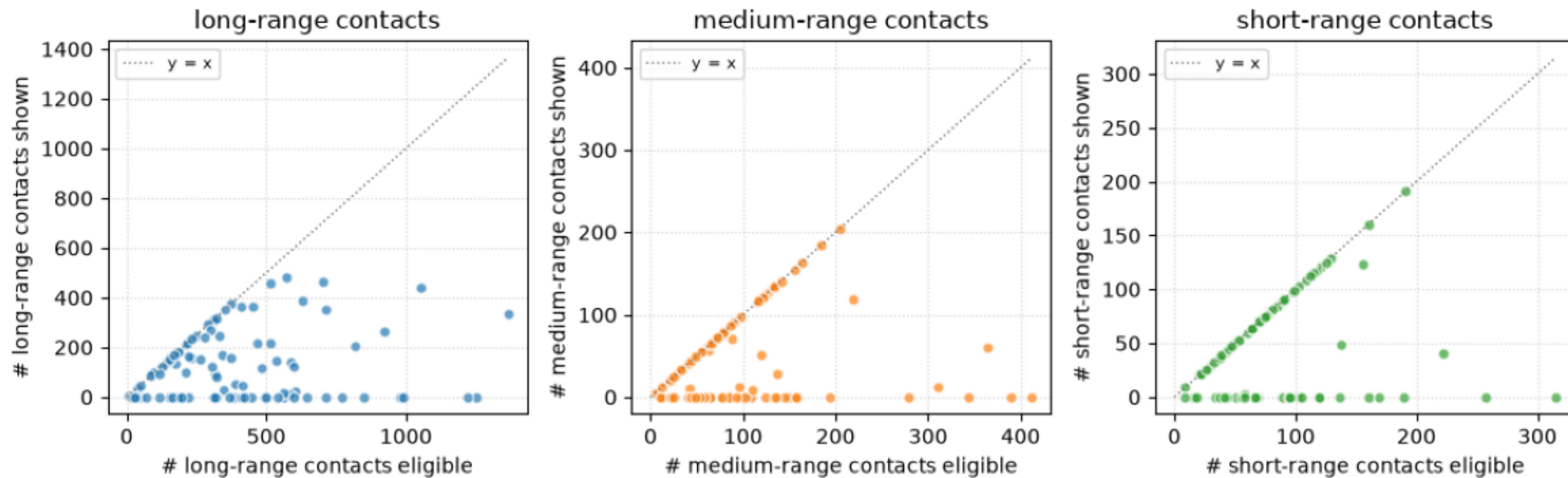
07_think_distribution.png

Three views of think-token cost. Left: total <think> tokens per doc (mean ~8). Middle: initial-run length k_1 when the $P=0.75$ gate fires — distribution matches $\text{Geom}(0.13)$. Right: # additional inter-statement runs — 0.3 (matches $\text{int}(\text{Uniform}(-4, 4))$ clamped to 0).



08_eligible_vs_shown.png

Per-mode: x = eligible contacts in the protein, y = contacts placed in the doc. Points on $y=x$ mean the doc captured *every* available contact of that mode. Medium- and short-range proteins frequently saturate; long-range proteins routinely have far more eligible contacts than the per-mode budget can fit.



09_eligible_pool_vs_length.png

Eligible-contact pool size by mode versus sequence length. Long-range pools grow super-linearly with length ($\sim N^{1.5}$ visually on this log-log) while medium- and short-range pools grow roughly linearly. This is what makes long-range the bottleneck on fraction-captured.

